Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul Calculatoare și Tehnologia Informației

Computer Architecture
Laborator 0: Recapitulare

Ștefan-Dan Ciocîrlan · 2026-2027

Cuprins

1. Circuite combinaţionale 1
2. Porţi logice 2
   2.1. Multiplexorul 4:1 3
3. Sumatoare 6
   3.1. Sumatorul elementar 6
   3.2. Sumatorul elementar parţial 7
   3.3. Sumatorul elementar complet 8
   3.4. Sumatorul cu transport succesiv 8
4. Circuite secvenţiale 10
5. Bistabilul D 10
6. Automate finite 12
   6.1. Recunoaşterea secvenţei „ba” 12
   6.2. Intersecţie semaforizată 15

1. Circuite combinaționale

Circuitele logice combinaționale aplică funcții logice pe semnalele de intrare pentru a obține semnalele de ieșire. Valorile de ieșire depind doar de valorile de intrare, iar când starea unei intrări se schimbă, acest lucru se reflectă imediat la ieșirile circuitului.

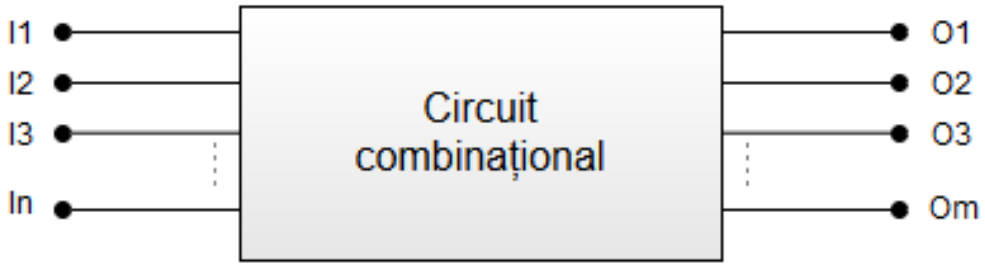


Figura 1: Diagrama bloc pentru un circuit combinațional cu n intrări și m ieșiri

Logica combinațională poate fi reprezentată prin:
- diagrame structurale la nivel de porți logice
- tabele de adevăr
- expresii booleene (funcții logice)

Circuitele combinaționale sunt folosite în procesoare în cadrul componentelor de calcul, iar cele mai des întâlnite sunt:
- multiplexoarele și demultiplexoarele

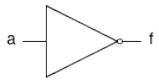
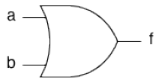
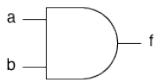
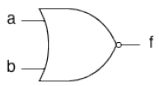
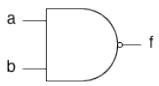
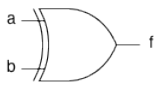
- codificatoarele şi decodificatoarele
- sumatoarele
- comparatoarele
- memoriile ROM (read-only, nu păstrează stare)

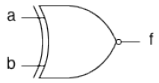
Un exemplu de folosire a sumatoarelor este în cadrul Unităților Aritmetice-Logice (UAL) din interiorul procesoarelor.

2. Porți logice

Porțile logice reprezintă componentele de bază disponibile în realizarea circuitelor combinaționale. Ele oglindesc operațiile din algebra booleană, algebră care stă la baza teoriei circuitelor combinaționale. În sunt prezentate cele mai întâlnite porți logice împreună cu operația booleană pe care o implementează.

Tabelul 1: Tabel de adevăr pentru porti logice elementare

Denumire	Simbol	Operator	a	b	f
Inversor (NOT)		f = !a	0		1
			1		0
Poarta SAU (OR)		f = a || b	0	0	0
			0	1	1
			1	0	1
			1	1	1
Poarta ŞI (AND)		f = a && b	0	0	0
			0	1	0
			1	0	0
			1	1	1
Poarta SAU-NU (NOR)		f = !(a || b)	0	0	1
			0	1	0
			1	0	0
			1	1	0
Poarta ŞI-NU (NAND)		f = !(a && b)	0	0	1
			0	1	1
			1	0	1
			1	1	0
Poarta SAU EXCLUSIV (XOR)		f = a ^ b	0	0	0
			0	1	1

Denumire	Simbol	Operator	a	b	f
			1	0	1
			1	1	0
Poarta SAU EXCLUSIV NU (XNOR)		f = !(a ^ b)	0	0	1
			0	1	0
			1	0	0
			1	1	1

2.1. Multiplexorul 4:1

Un multiplexor digital este un circuit combinațional care implementează o funcție de selecție a uneia dintre intrările sale.

- 2^n intrări
- n intrări de selecție
- o ieșire

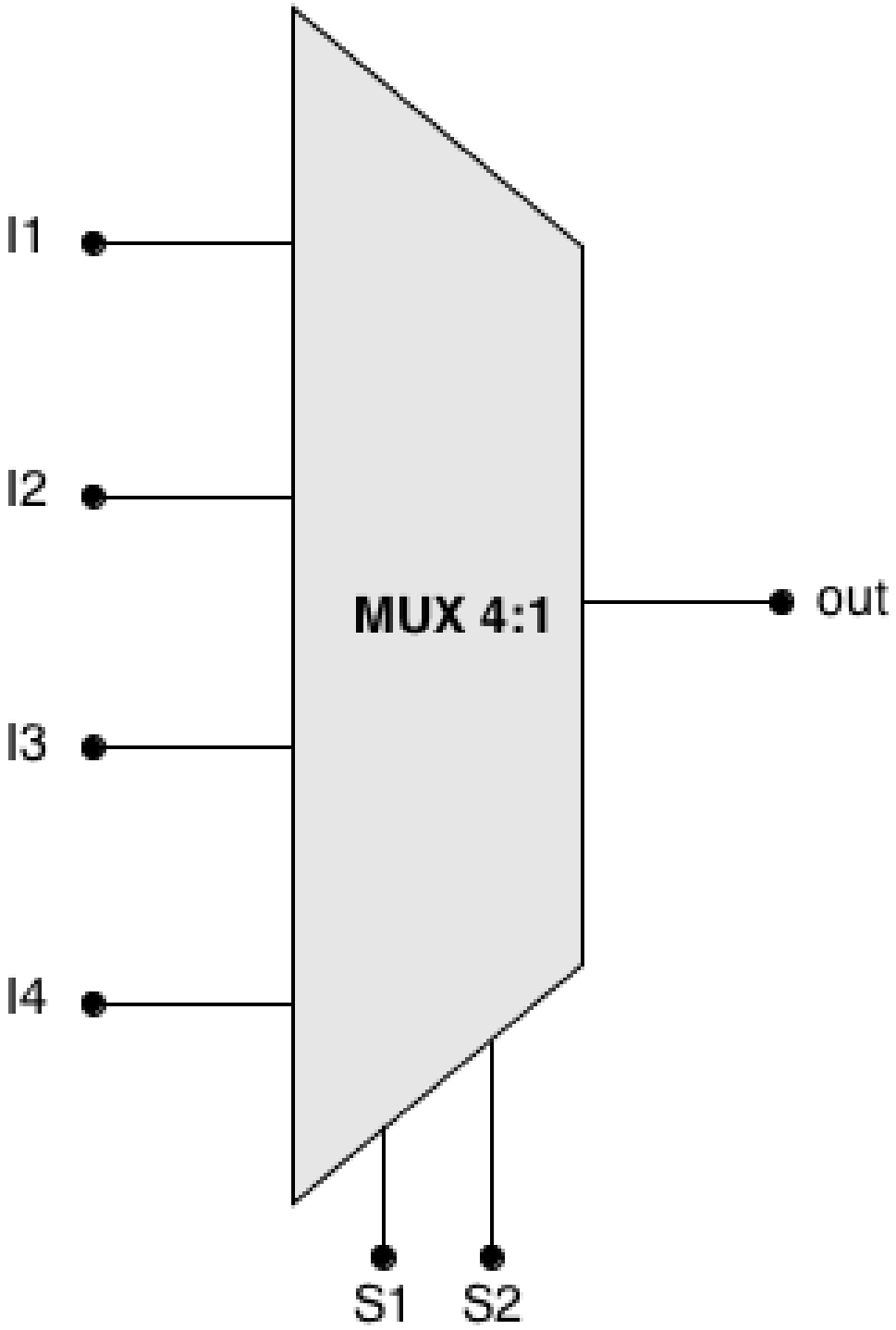


Figura 2: Diagrama bloc a multiplexorului 4:1

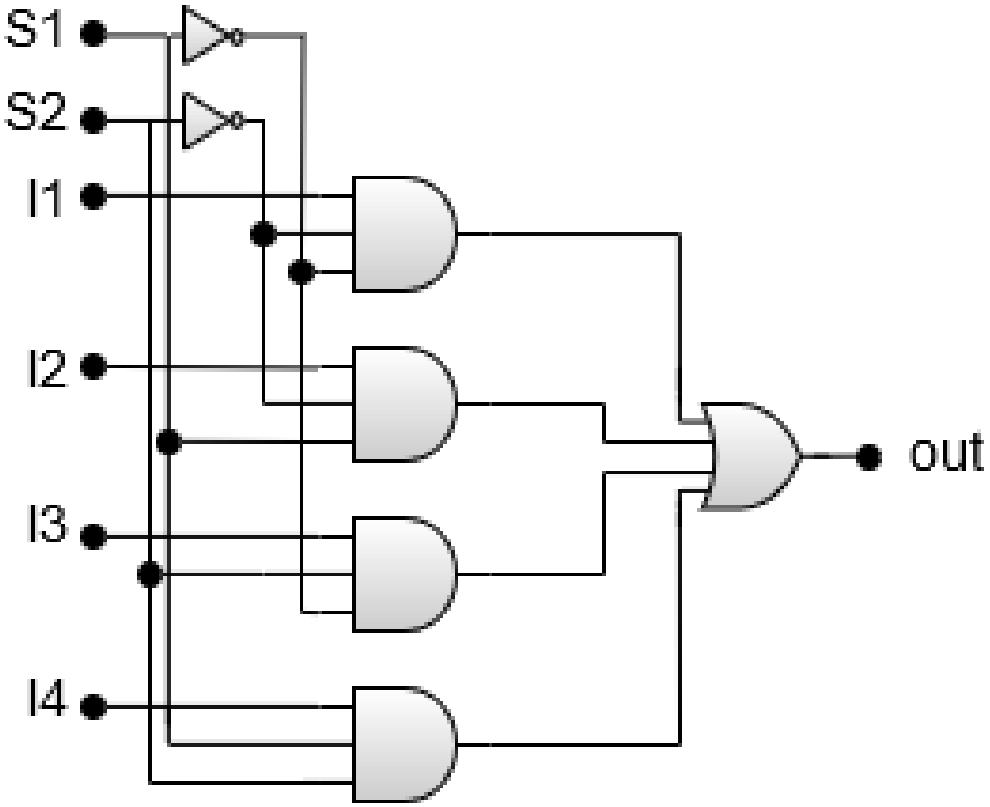


Figura 3: Schema logică a multiplexorului 4:1

Alegerea semnalului de ieșire se face pe baza intrărilor de selecție, care reprezintă în baza 2 numărul intrării ce trebuie selectate. În exemplul din imaginea de mai sus avem schema bloc a unui multiplexor cu 4 intrări, iar acesta are nevoie de două intrări de selecție.

Funcția inversă a multiplexorului este realizată de către circuitele de demultiplexare, care preiau un semnal de intrare și folosesc intrările de selecție pentru a-l transmite pe una din ieșirile posibile.

Tabelul 2: Tabel de adevăr pentru multiplexorul 4:1

S2	S1	out
0	0	I1
0	1	I2
1	0	I3
1	1	I4

Deoarece multiplexorul 4:1 are 6 intrări, tabelul de adevăr devine destul de mare și nu mai este indicat de pornit de la acesta pentru obținerea funcției logice. Din descrierea funcționării circuitului și proprietățile porții AND, putem deduce termenii formulei:

$$f = \overline{s_1} \cdot \overline{s_2} \cdot I_1 + s_1 \cdot \overline{s_2} \cdot I_2 + \overline{s_1} \cdot s_2 \cdot I_3 + s_1 \cdot s_2 \cdot I_4$$

Conform formulei se poate realiza circuitul cu porți logice din imaginea de mai sus.

```
int mux4to1(int s0, int s1, int i0, int i1, int i2, int i3) {
    // Use the select value to choose the appropriate input
    switch ((s1 << 1) | s0) {  // Combine s0 and s1 into a 2-bit value
        case 0:
            return i0; // If s1 and s0 are both 0, return input i0
        case 1:
            return i1;
        case 2:
            return i2;
        case 3:
```

```
            return i3;
        default:
            return 0; // Default case (should never be reached)
    }
}
```

Codul sursa pentru functia mux4to1

3. Sumatoare

3.1. Sumatorul elementar

Sumatoarele (*adders*), folosite cel mai mult în unitățile aritmetice logice ale procesoarelor, realizează adunări pe un număr dat de biți, furnizând la ieșirea circuitului suma și transportul (*carry*) rezultat în urma operației.

Există mai multe tipuri de sumatoare pentru adunarea numerelor pe *n* biți, iar acestea se bazează pe sumatoare simple de 1 bit, care pot fi de două tipuri:

- sumatorul elementar parțial (*Half adder*) - însumează doi operanzi pe 1 bit și oferă la ieșire suma acestora și transportul.
- sumatorul elementar complet (*Full adder*) - însumează doi operanzi pe 1 bit și un transport și oferă la ieșire suma acestora și transportul.

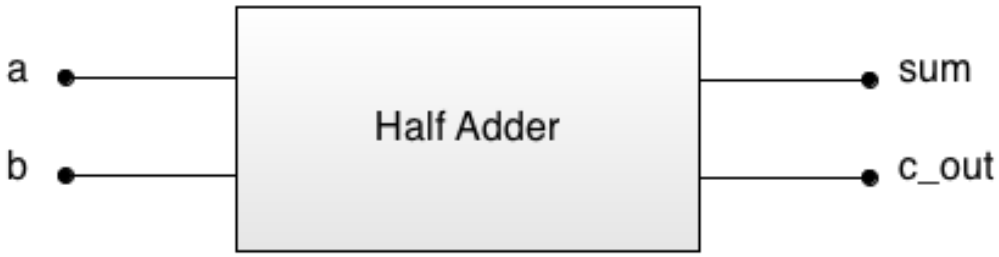


Figura 4: Diagrama bloc pentru half adder

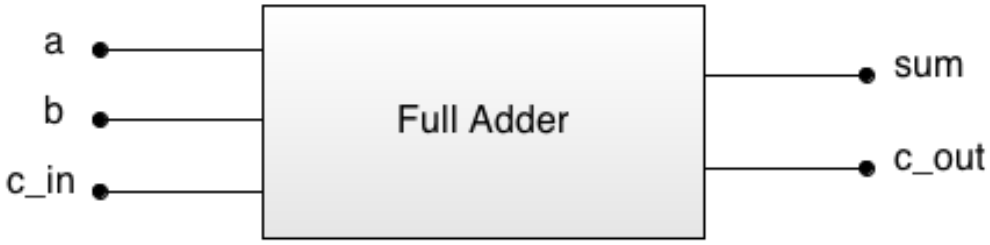


Figura 5: Diagrama bloc pentru full adder



Figura 6: Diagrama semnale pentru full adder

3.2. Sumatorul elementar parțial

Acest sumator este în continuare descris prin expresiile booleene, tabelul de adevăr și schema logică.

Tabelul 3: Tabelul de adevăr pentru sumatorul elementar parțial

a	b	sum	c_out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Din tabelul de adevăr se pot deduce următoarele formule:

```
sum   = a ^  b
c_out = a && b
```

Conform acestor formule putem exprima circuitul prin porți logice, ca în imaginea de mai jos:

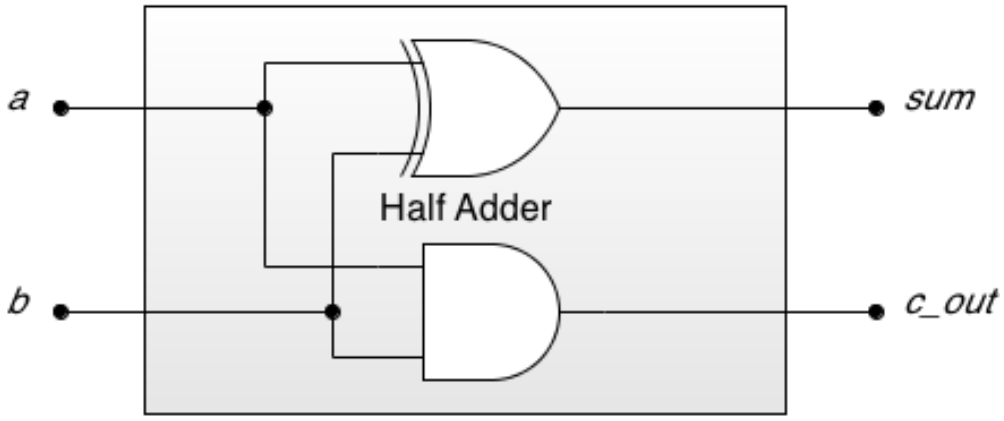


Figura 7: Schema logică pentru half adder

Dintr-un tabel de adevăr, pentru fiecare output se va deduce o funcție/expresie aplicând următoarele reguli:

- fiecare rând din tabel pentru care funcția are valoarea 1 va genera un termen
- termenii sunt formați din parametrii funcției legați prin ȘI
  - dacă parametrul are valoarea 1 se consideră în formă directă
  - dacă parametrul are valoarea 0 se consideră în formă negată
- se aplică SAU între toți termenii deduși

Pentru sumatorului elementar parțial avem:

$\text{sum} = \overline{a} \cdot b + a \cdot \overline{b}$

$\text{carry} = a \cdot b$

```
void half_adder(const int a, const int b, int &sum, int &carry) {
    sum = a ^ b;       // Sum is the XOR of the inputs
    carry = a & b;     // Carry is the AND of the inputs
}
```

Codul sursa pentru functia half_adder

În multe cazuri aceste formule sunt prea complexe, conținând multe operații și necesitând multe porți logice pentru a fi implementate. Pentru a reduce complexitatea formulelor rezultate se poate aplica un procedeu de **minimizare**, care va reduce dimensiunea termenilor sau chiar îi va elimina. Minimizarea se poate realiza folosind teoremele algebrei booleene sau grafic, prin diagrame Karnaugh.

3.3. Sumatorul elementar complet

Tabelul 4: Tabelul de adevăr pentru full adder

a	b	c_in	sum	c_out
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Din tabelul de adevăr se pot deduce următoarele formule:

```
sum   =   a ^ b  ^  c_in
c_out = ((a ^ b) && c_in) || (a && b)
```

Conform acestor formule putem exprima circuitul prin porți logice sau putem folosi sumatoare elementare parțiale, ca în imaginea de mai jos:

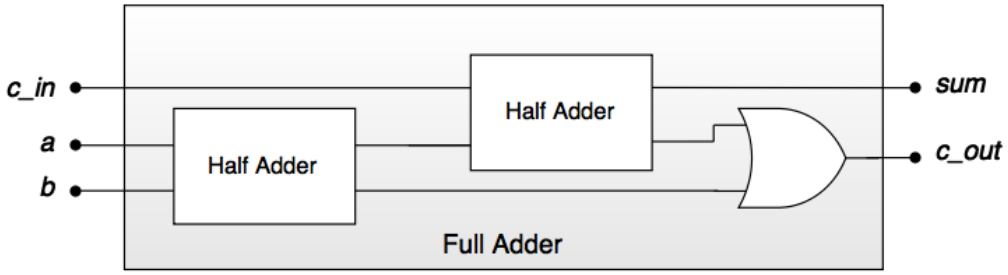


Figura 8: Schema logică pentru full adder

```
void full_adder(const int a, const int b, const int cin, int &sum, int &carry)
{
    sum   =   a ^ b  ^  c_in;
    carry = ((a ^ b) && c_in) || (a && b);
}
```

Sau utilizand sumatoare elementare partiale:

```
void full_adder(const int a, const int b, const int cin, int &sum, int &carry) {
    int sum1, carry1, carry2;

    // First half adder
    half_adder(a, b, sum1, carry1);

    // Second half adder
    half_adder(sum1, cin, sum, carry2);

    // Final carry
    carry = carry1 | carry2;
}
```

Codul sursa pentru functia full_adder

3.4. Sumatorul cu transport succesiv

Cel mai intuitiv mod de a forma un sumator este de a lega în cascadă mai multe sumatoare elementare complete pe 1 bit. În acest fel se formează un sumator cu transport succesiv (eng. *ripple-*

carry adder), cum este cel pe 4 biți din imaginea de mai jos, care primește la intrare `a[3:0]`, `b[3:0]`, `c_in` și are ca ieșiri suma `s[3:0]` și transportul `c_out`. În cazul sumatoarelor pe mai mulți biți nu mai este indicat de pornit întâi de la o tabelă de adevăr deoarece aceasta ajunge la dimensiuni prea mari.

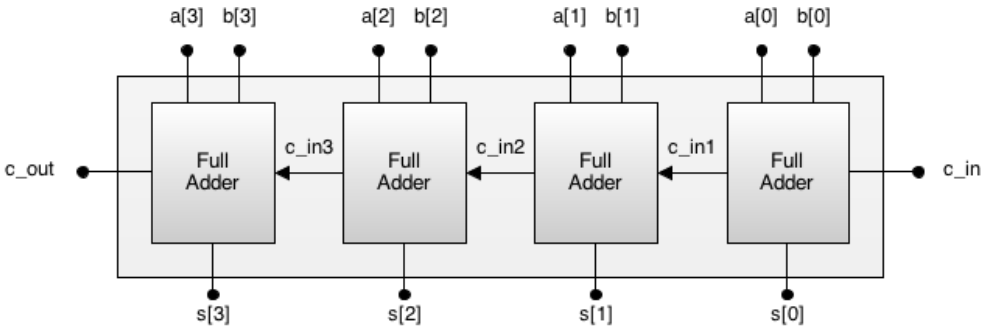


Figura 9: Schema sumatorului cu transport succesiv, pe 4 biți

```
template <int N>
void ripple_adder(const int (&a)[N], const int (&b)[N], int (&sum)[N], const int carry_in, int &carry_out) {
    // Initialize carry to 0
    int carry_tmp[N+1];
    carry_tmp[0] = carry_in;

    // Iterate over each bit
    for (int i = 0; i < N; ++i) {
        // Calculate the sum and carry for the current bit
        full_adder(a[i], b[i], carry_tmp[i], sum[i], carry_tmp[i+1]);
    }

    // Set the final carry
    carry_out = carry_tmp[N];
}
```

Codul sursa pentru functia ripple_adder

Un alt avantaj al acestui design simplu, este că se pot forma sumatoare pe mai mulți biți din înlănțuirea oricâtor sumatoare. De exemplu, pentru a însuma numere pe 16 biți se poate crea un sumator ripple-carry din legarea în cascadă a 4 sumatoare pe 4 biți, ca în imaginea de mai jos.

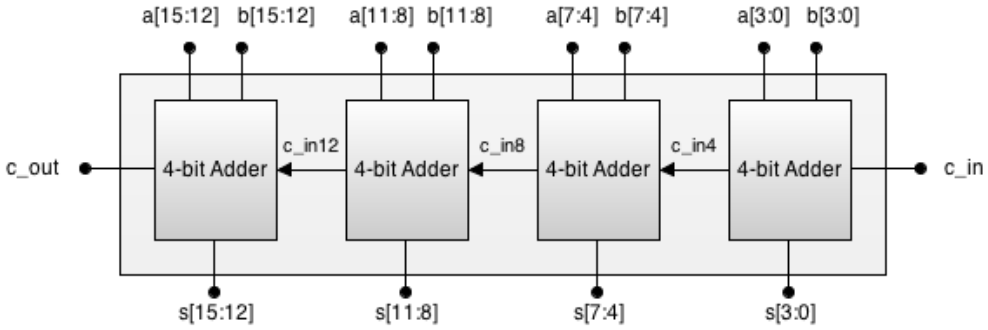


Figura 10: Schema sumatorului cu transport succesiv, pe 16 biți

Deși are un design simplu, dezavantajul acestui sumator este că este **lent**, fiecare sumator elementar necesitând transportul de la sumatorul precedent. Există alte sumatoare, cum ar fi cel cu transport anticipat (eng. Carry-lookahead adder), care oferă o funcționare mai rapidă, eliminând așteptarea propagării transportului.

```
// Example for 16-bit ripple adder using four 4-bit ripple adders
const int M = 16;
int a16[M] = {1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0}; // Example 16-bit binary number
int b16[M] = {1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1}; // Example 16-bit binary number
int sum16[M];
int carry16[5] = {0}; // Carry array for each 4-bit adder

for (int i = 0; i < M; i += 4) {
    ripple_adder<4>(*(int(*)[4])(a16 + i), *(int(*)[4])(b16 + i), *(int(*)[4])(sum16 + i), carry16[i / 4], carry16[i / 4 + 1]);
}
```

Codul sursa

4. Circuite secvențiale

Spre deosebire de circuitele logice combinaționale, cele secvențiale (eng: *sequential logic*) nu mai depind exclusiv de valoarea curentă a intrărilor, ci și de stările anterioare ale circuitului.

Logica secvențială poate fi de două tipuri: **sincronă** și asincronă. În primul caz, cel cu care vom lucra și la laborator, este folosit un semnal de ceas care comandă elementul/elementele de memorare, acestea schimbându-și starea doar la impulsurile de ceas. În al doilea caz, ieșirile se modifică atunci când se modifică și intrările, neexistând un semnal de ceas pentru elementele de memorare. Circuitele secvențiale asincrone sunt mai greu de proiectat deoarece pot apărea probleme de sincronizare. Din această cauză ele sunt folosite mai rar.

În continuare ne vom referi doar la circuitele secvențiale sincrone.

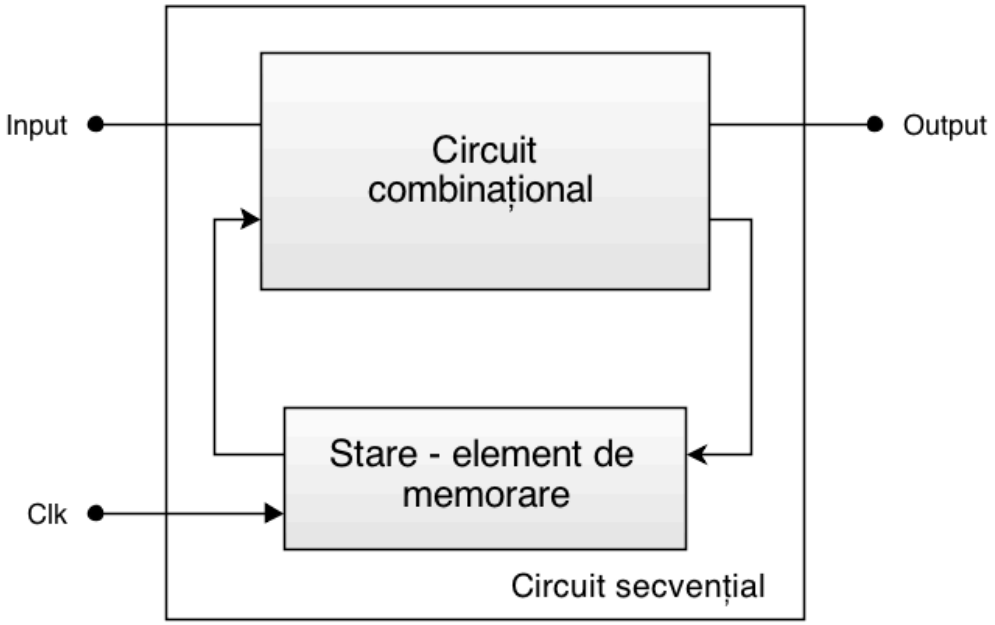


Figura 11: Schema bloc a unui circuit secvențial sincron

5. Bistabilul D

Elementele de memorare din circuitele secvențiale pot fi implementate prin bistabile (eng. *flip-flops*). Acestea stochează valori în funcție de valoarea de la intrare și de semnalul de ceas. Valoarea stocată poate fi schimbată doar atunci când ceasul realizează o tranziție activă (un semnal de ceas poate fi „activ” pe front crescător (eng. *rising edge*) sau pe front descrescător (eng. *falling edge*)).

Există 4 tipuri principale de bistabile: D, T, SR și JK, iar în acest laborator ne vom axa pe bistabilul D. Acesta are un design simplu și este folosit în general pentru implementarea registrelor din procesoare (cea mai mică și mai rapidă unitate de stocare din ierarhia de memorie).

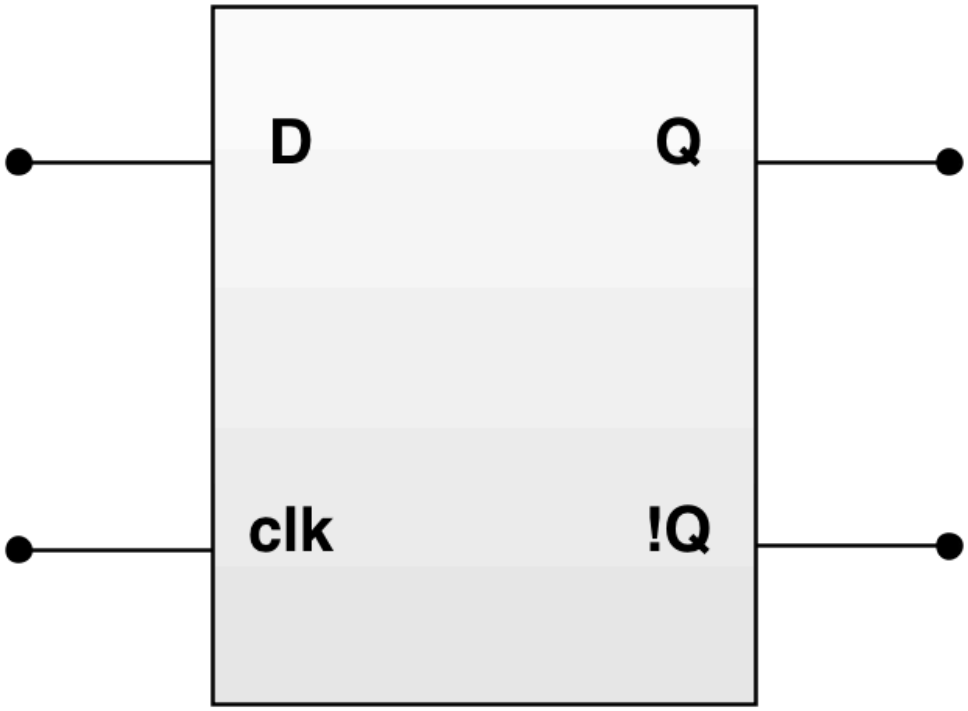


Figura 12: Diagrama bloc pentru bistabilul D

Intrările și ieșirile circuitului sunt:

- ’’D’’ - valoarea (*data*) de stocat
- ’’clk’’ - semnalul de ceas, considerat activ pe front crescător în descrierile următoare
- ’’Q’’ - starea curentă
- ’’!Q’’ - starea curentă negată

Ca mod de funcționare, ecuația caracteristică a sa este ’’Qnext = D’’, adică starea următoare (’’Qnext’’) a bistabilului depinde doar de intrarea ’’D’’, fiind independentă de starea curentă (’’Q’’), după cum se observă și din tabelul de mai jos.

Tabelul 5: Tabelul de tranziții pentru bistabilul D

D	Q	Qnext
0	0	0
0	1	0
1	0	1
1	1	1

Pentru a înțelege mai ușor comportamentul bistabilelor, pe lângă tabelele de tranziții mai sunt utile și diagramele de semnale (eng. *timing diagrams*), cum este cea din figura de mai jos, unde se poate observa cum ieșirea ’’Q’’ se schimbă doar pe frontul crescător de ceas și devine egală cu intrarea ’’D’’ în momentul tranziției ceasului.

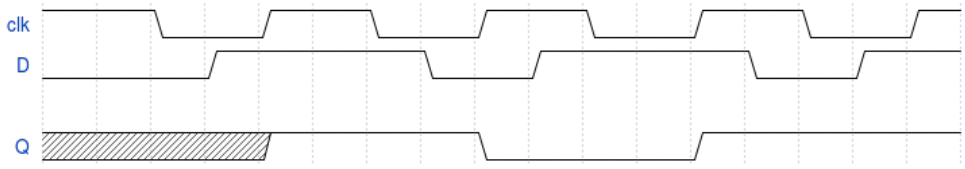


Figura 13: Diagrama de semnale pentru bistabilul D

6. Automate finite

Prin automate finite (eng. *Finite-state machine - FSM*) înțelegem de fapt un circuit secvențial sincron așa cum a fost el descris anterior. De obicei, proiectarea unui automat finit pornește de la o descriere informală a modului în care automatul trebuie să funcționeze. Primul pas în realizarea automatului este descrierea formală a funcționării acestuia. Două dintre metodele prin care un automat finit poate fi descris sistematic sunt:

- **Diagrama de stări** prezintă într-un mod grafic funcționarea unui automat finit. Stările automatului sunt reprezentate prin noduri, iar tranzițiile sunt reprezentate prin arce între starea sursă și starea destinație. Fiecare arc este marcat cu condiția necesară pentru a fi efectuată o tranziție. De asemenea, eventualele semnale de ieșire ale automatului sunt marcate în dreptul stărilor care generează acele ieșiri.

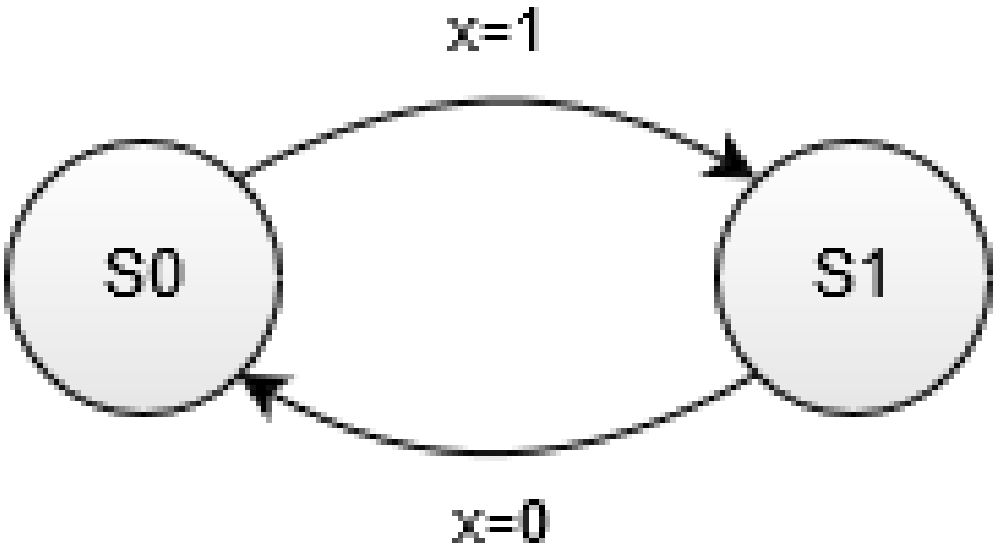


Figura 14: Exemplu de diagramă de stări

- **Tabelul de tranziţii** prezintă funcționarea unui automat finit sub formă de tabel. Fiecare rând al tabelului reprezintă o tranziție a automatului și conține starea curentă, starea următoare și intrările necesare pentru a activa tranziția.

Tabelul 6: Exemplu de tabel de tranziții

Starea curentă	x	Starea următoare
S0	1	S1
S1	0	S0

În continuare vom proiecta două automate finite simple:

6.1. Recunoaşterea secvenţei „ba”

Se dorește proiectarea unui automat finit capabil să recunoască secvența „ba”. Automatul primește la intrare în mod continuu caractere codificate printr-un semnal de un bit (caracterele posibile sunt „a” și „b”). Ieșirea automatului va consta dintr-un semnal care va fi activat (valoarea 1) atunci când ultimele două caractere introduse vor fi „b” urmat de „a”. Semnalul de ieșire va rămâne activ până la introducerea unui nou caracter, după care automatul va continua operația de recunoaștere.

Vom începe proiectarea automatului prin identificarea intrărilor și ieșirilor. Din descriere observăm că intrarea este formată dintr-un singur semnal de 1 bit (automatul va avea și o intrare de ceas, însă aceasta nu este considerată intrare propriu zisă de date). Deoarece codificarea caracterelor nu este specificată vom presupune că valoarea 0 indică un caracter „a”, iar valoarea 1 indică un caracter

„b”. Ieşirea este formată deasemenea dintr-un semnal de 1 bit cu valoarea 1 atunci când secvența căutată a fost găsită şi 0 în rest.

Vom realiza în continuare diagrama de stări a automatului. La pornire, vom iniţializa automatul într-o stare pe care o vom numi ’’S0′’. Dacă la prima tranziție de ceas intrarea are valoarea:

- 0 (caracterul „a”) - vom avansa într-o stare pe care o vom numi ’’Sa’’ care ne spune că intrarea precedentă a fost „a”
- 1 (caracterul „b”) - vom avansa într-o stare pe care o vom numi ’’Sb’’ care ne spune că intrarea precedentă a fost „b”

În continuare vom analiza ce se întâmplă atunci când automatul este în starea ’’Sa’’. Dacă la intrare avem valoarea:

- 0 (caracterul „a”) - automatul va rămâne în acestă stare, care ne spune că intrarea precedentă a fost „a”
- 1 (caracterul „b”) - automatul va trece în ’’Sb’’, care ne spune că intrarea precedentă a fost „b”

Dacă ne aflăm în starea ’’Sb’’ şi automatul primeşte la intrare valoarea:

- 0 (caracterul „a”) - automatul a întâlnit secvența dorită „ba” (fiind în starea ’’Sb’’ intrarea precedentă a fost „b”, iar intrarea curentă este „a”); vom avansa într-o stare pe care o vom numi ’’SA’’ în care vom activa ieşirea automatului; de asemenea, această stare ne spune şi că intrarea precedentă a fost „a”, lucru folosit pentru a putea recunoaşte şi următoarele secvențe „ba” care vor mai fi întâlnite la intrare
- 1 (caracterul „b”) - automatul va rămâne în această stare, care ne spune că intrarea precedentă a fost „b”

Dacă ne aflăm în starea ’’SA’’ şi automatul primeşte la intrare valoarea:

- 0 (caracterul „a”) - automatul va trece în starea ’’Sa’’ care ne spune că intrarea precedentă a fost „a”, însă nu vom activa ieşirea automatului deoarece automatul nu a văzut şi caracterul „b”
- 1 (caracterul „b”) - automatul va trece în starea ’’Sb’’ care ne spune că intrarea precedentă a fost „b”

În momentul de faţă comportamentul automatului a fost descris complet, toate cele 4 stări identificate având definite tranzițiile pentru toate combinațiile semnalelor de intrare. Figura de mai jos prezintă diagrama de stări a automatului.

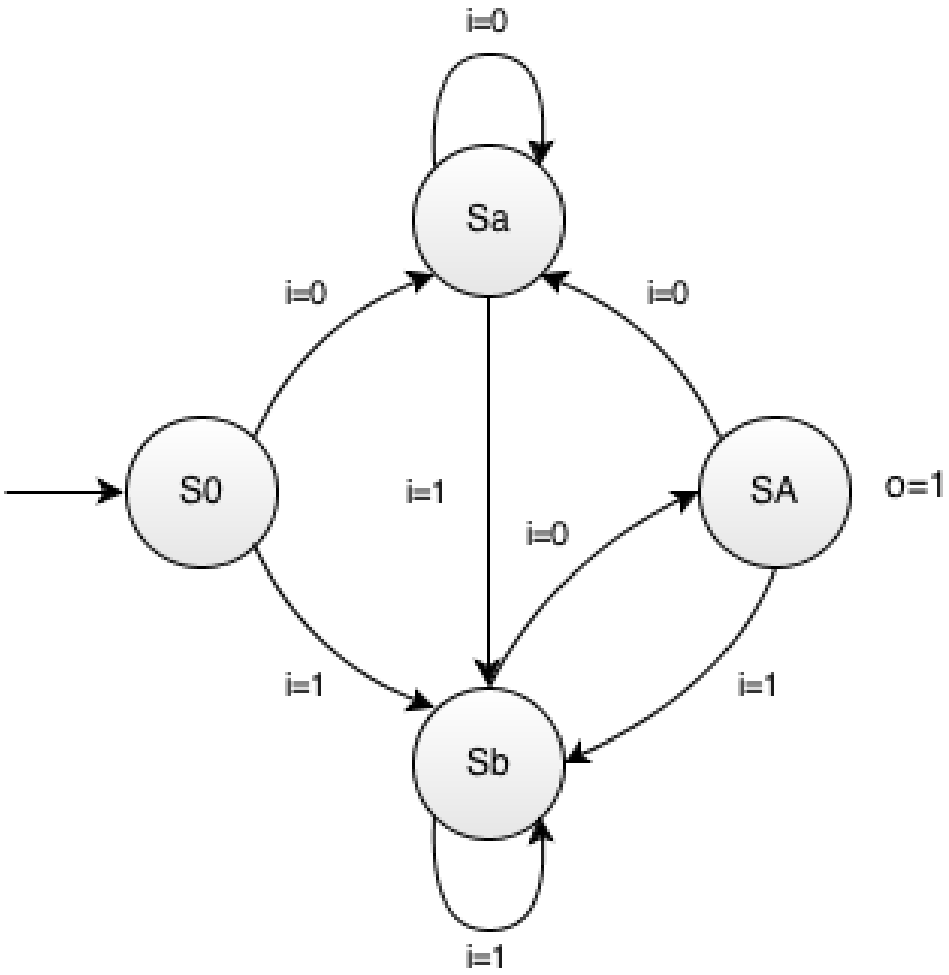


Figura 15: Automatul de recunoaștere a secvenței „ba”

O dată determinată diagrama de stări a automatului, putem trece la implementarea acestuia într-un limbaj cunoscut (C/C++/C#/Java):

```
void FSM_ba() {
  // FSM input: 0 - a, 1 - b, other - exit
  // FSM output: 0 - not found, 1 - found
  // FSM state: 0 - S0, 1 - Sa, 2 - Sb, 3 - SA
  int state = 0;
  int in, out;
  out = 0;
  in = 0;

  while(1) {
      switch(state) {
          case 0:
              out = 0;
              break;

          case 1:
              out = 0;
              break;

          case 2:
              out = 0;
              break;

          case 3:
```

```
            out = 1;
            break;
      }
    std::cout << "Output: " << out << std::endl;

    // Read input
    std::cout << "Please input a number (0 for a, 1 for b, other exit ): ";
    std::cin >> in;
    if (in > 1) {
        break;
    }

      switch(state) {
          case 0:
              if(in == 0)
                  state = 1;
              else
                  state = 2;
              break;

          case 1:
              if(in == 0)
                  state = 1;
              else
                  state = 2;
              break;

          case 2:
              if(in == 0)
                  state = 3;
              else
                  state = 2;
              break;

          case 3:
              if(in == 0)
                  state = 1;
              else
                  state = 2;
              break;
      }
  }
}
```

Codul sursa pentru functia FSM_ba

6.2. Intersecţie semaforizată

Se dorește modelarea prin intermediul unui automat de stări a unei intersecții semaforizate în care mașinile pot intra din nord (N), est (E), sud(S) sau vest (W). Semaforul din nord este sincronizat cu semaforul din sud, iar cel din est este sincronizat cu cel din vest. Duratele de timp pentru cele două direcții vor fi: Nord - Sud: roșu - 40 sec, galben - 10sec, verde - 50sec; Est-Vest: roșu - 60 sec, galben - 10 sec, verde - 30 sec.

Vom începe proiectarea automatului prin identificarea intrărilor şi ieşirilor. Deoarece descrierea informală nu conţine informaţii despre intrările şi ieşirile necesare vom folosi oricâte intrări şi ieşiri avem nevoie pentru implementarea comportamentului. Un minim de ieşiri pentru automat

reprezintă semnalele de comandă pentru culorile semaforului pentru pietoni şi pentru maşini. Cele 5 semnale vor fi:

- N_rosu- aprindere culoare roşie pentru maşinile din Nord
- N_galben - aprindere culoare galbenă pentru maşinile din Nord
- N_verde - aprindere culoare verde pentru maşinile din Nord
- E_rosu- aprindere culoare roşie pentru maşinile din Est
- E_galben - aprindere culoare galbenă pentru maşinile din Est
- E_verde - aprindere culoare verde pentru maşinile din Est
- S_rosu- aprindere culoare roşie pentru maşinile din Sud
- S_galben - aprindere culoare galbenă pentru maşinile din Sud
- S_verde - aprindere culoare verde pentru maşinile din Sud
- W_rosu- aprindere culoare roşie pentru maşinile din Vest
- W_galben - aprindere culoare galbenă pentru maşinile din Vest
- W_verde - aprindere culoare verde pentru maşinile din Vest

Pentru a măsura duratele de timp am putea folosi semnalul de ceas al automatului, introducând multiple stări cu tranziţii necondiţionate, în care o culoare a semaforului este ţinută aprinsă. Având în vedere însă că semnalul de ceas pentru un automat are o perioadă de ceas mică (%%<<%% 1 sec) am avea nevoie de multe stări pentru a realiza o durată de 30 sec. O soluţie mult mai bună este să folosim un numărător pentru a realiza întârzierile necesare. Numărătorul este un circuit secvenţial (automat finit) care poate număra crescător sau descrescător tranziţiile unui semnal, având un semnal de ieşire care este activat atunci când indexul ajunge la 0 sau la o valoare care poate fi controlată. Concret, pentru măsurarea duratelor de timp în automatul nostru vom folosi un numărător crescător a cărui valoare maximă o vom configura pentru a obţine duratele de timp necesare, în funcţie de perioada de ceas a automatului. Vom adăuga astfel o ieşire (''T''), care va controla valoarea maximă a numărătorului şi o intrare (''done'') care va primi semnalul de terminare de la numărător.

Diagrama de stări a automatului va urmări tranziţia celor 3 culori ale semaforului pentru maşini: verde -> galben -> roşu -> verde .

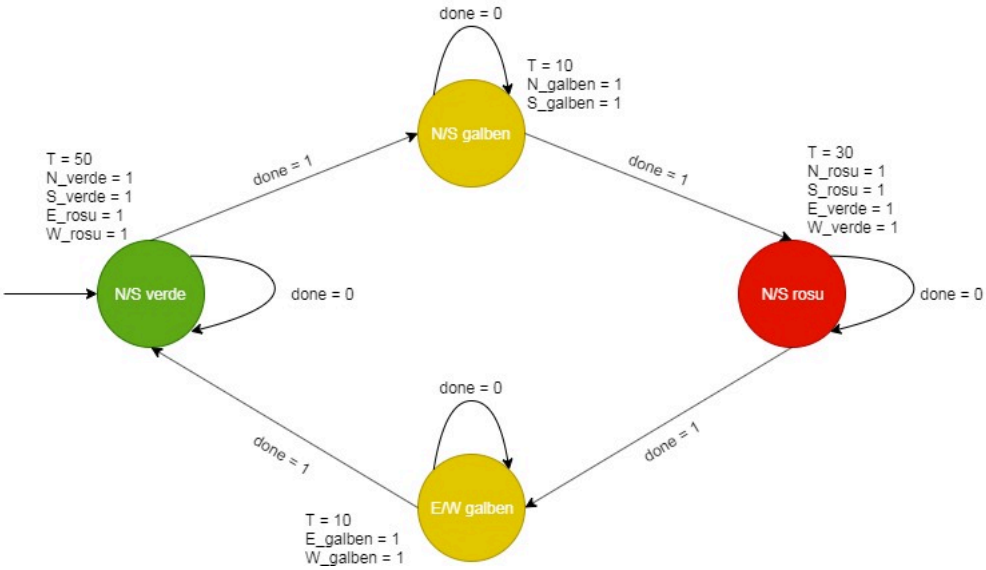


Figura 16: Automatul intersecţiei

Odată determinată diagrama de stări a automatului, putem trece la implementarea acestuia într-un limbaj cunoscut (C/C++/C#/Java):

```
void FSM_intersectie() {
    int state = 0; // FSM state: 0 - N/S_verde, 1 - N/S_galben, 2 - N/S_rosu, 3 - E/W_galben
```

```
int N_rosu;
int N_galben;
int N_verde;
int S_rosu;
int S_galben;
int S_verde;
int W_rosu;
int W_galben;
int W_verde;
int E_rosu;
int E_galben;
int E_verde;
int done;
int T;
done = 0;
T = 0;

while(1) {
    N_rosu = 0;
    N_galben = 0;
    N_verde = 0;
    S_rosu = 0;
    S_galben = 0;
    S_verde = 0;
    W_rosu = 0;
    W_galben = 0;
    W_verde = 0;
    E_rosu = 0;
    E_galben = 0;
    E_verde = 0;
    switch(state) {
        case 0:
            E_rosu = 1;
            W_rosu = 1;
            N_verde = 1;
            S_verde = 1;
            T = 50;
            if(done == 1)
                state = 1;
            else
                state = 0;
            break;
        case 1:
            N_galben = 1;
            S_galben = 1;
            E_rosu = 1;
            W_rosu = 1;
            T = 10;
            if(done == 1)
                state = 2;
            else
                state = 1;
            break;
        case 2:
            N_rosu = 1;
            S_rosu = 1;
            E_verde = 1;
            W_verde = 1;
            T = 30;
            if(done == 1)
                state = 3;
```

```
                else
                    state = 2;
                break;
            case 3:
                N_rosu = 1;
                S_rosu = 1;
                E_galben = 1;
                W_galben = 1;
                T = 10;
                if(done == 1)
                    state = 0;
                else
                    state = 3;
                break;
        }

        std::cout << "Stare: " << state << std::endl;
        std::cout << "Done: " << done << std::endl;
        std::cout << "N: " << N_rosu << N_galben << N_verde << std::endl;
        std::cout << "S: " << S_rosu << S_galben << S_verde << std::endl;
        std::cout << "W: " << W_rosu << W_galben << W_verde << std::endl;
        std::cout << "E: " << E_rosu << E_galben << E_verde << std::endl;
        std::cout << "Timp: " << T << std::endl;
        std::cin >> done;
        if (done > 1) {
            break;
        }
    }
}
```

Codul sursa pentru functia FSM_intersectie